This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

DropGaussian: Structural Regularization for Sparse-view Gaussian Splatting

Hyunwoo Park Gun Ryu Wonjun Kim*
Konkuk University
{pzls, fbrjs15, wonjkim}@konkuk.ac.kr

Abstract

Recently, 3D Gaussian splatting (3DGS) has gained considerable attentions in the field of novel view synthesis due to its fast performance while yielding the excellent image quality. However, 3DGS in sparse-view settings (e.g., three-view inputs) often faces with the problem of overfitting to training views, which significantly drops the visual quality of novel view images. Many existing approaches have tackled this issue by using strong priors, such as 2D generative contextual information and external depth signals. In contrast, this paper introduces a prior-free method, so-called DropGaussian, with simple changes in 3D Gaussian splatting. Specifically, we randomly remove Gaussians during the training process in a similar way of dropout, which allows non-excluded Gaussians to have larger gradients while improving their visibility. This makes the remaining Gaussians to contribute more to the optimization process for rendering with sparse input views. Such simple operation effectively alleviates the overfitting problem and enhances the quality of novel view synthesis. By simply applying DropGaussian to the original 3DGS framework, we can achieve the competitive performance with existing prior-based 3DGS methods in sparse-view settings of benchmark datasets without any additional complexity. The code and model are publicly available at: https://github.com/DCVL-3D/DropGaussian_release.

1. Introduction

As the demand for realistic renderings and their applications increases rapidly, novel view synthesis (NVS) has become an essential technique. Recently, the neural radiance field (NeRF) [14] has been introduced in literature, which has a good ability to encode a given 3D scene into the implicit radiance field through learnable parameters of the neural network. Even though NeRF and its diverse variants have shown the remarkable performance, most previous methods require time-consuming processes for rendering as well as training. Meanwhile, a new technique, so-called 3D Gaussian splatting (3DGS) [8], has emerged in 2023 and is becoming a mainstream in the field of NVS. 3DGS is a method that generates an explicit radiance field, composed of a set of 3D Gaussians positioned in 3D space. Based on the property of point-based explicit representations, 3DGS demonstrates real-time operations for rendering while maintaining the rendering quality for novel view inputs. However, with sparse input views, 3DGS is still prone to overfitting due to insufficient cues for visual appearance and geometric layout in understanding a given 3D scene.

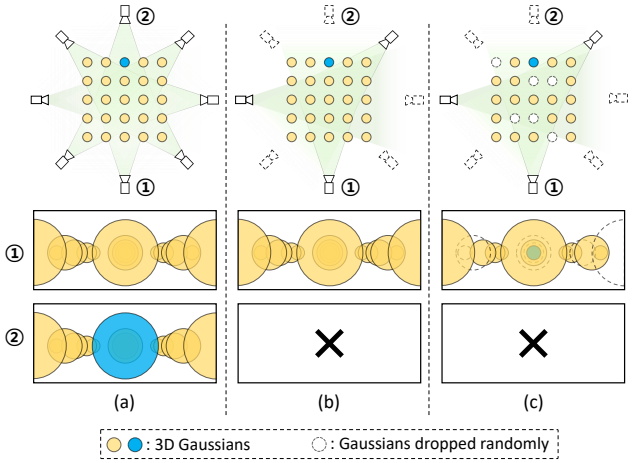


Figure 1. (a) Traditional settings of 3DGS. (b) Sparse-view set tings of 3DGS. (c) Effect of DropGaussian in sparse-view settings. The rendered outputs at each viewpoint are visualized in (a), (b), and (c), respectively.

Despite substantial efforts in the field of 3DGS, the optimization of 3D Gaussians only with sparse input views still remains challenging. To address this challenge, several approaches have begun to leverage the prior information of a given scene. For example, there have been meaningful attempts to adopt the result of monocular depth estimation as an external supervisory signal for imposing Gaussians on appropriate positions [5, 11, 36]. However, without considering the world coordinate, estimated depth scales vary across different views, which makes consistent regularization difficult. Even though the 2D generative contextual in-

*Corresponding author

formation also has been successfully employed to guide the rendering process to yield more realistic results [30], it requires high computational costs and often leads to unstable optimization due to the stochastic sampling process. On the other hand, the optical flow has been utilized to regularize the pixel-wise correspondence of 3D Gaussians between 3D Gaussians in sparse-view conditions [18]. Although such prior-based approaches have been actively explored to mitigate the overfitting problem driven by sparse input views, most of them have respective limitations such as error propagation and high computational burden.

In this paper, unlike previous methods relying on strong priors, we present a prior-free method that requires only a simple modification of 3DGS without additional computational costs. In conventional settings of 3DGS, Gaussians, which are far from the camera and probably occluded in a specific viewpoint, can be visible in other viewpoints as shown in Fig. 1 (a). In contrast, such Gaussians are often excluded from the field of view in sparse-view settings, resulting in receiving less gradient feedback due to the low visibility caused by occlusion between other Gaussians (see Fig. 1(b)). This ultimately leads to overfitting to a small number of training views. The key idea of the proposed method is to randomly remove Gaussians, so-called *DropGaussian*, during training instead of adopting the prior information for sparse input views. Based on our DropGaussian scheme, the remaining Gaussians are provided with the opportunity to be more visible as illustrated in Fig. 1 (c). Such simple operation makes the optimization process with sparse input views be more balanced in the sense that even less visible Gaussians receive adequate attentions during training (see Fig. 1(c)). Consequently, the model is able to figure out the whole layout of a given 3D scene in a comprehensive manner. This effectively generalizes the rendering performance to novel views and mitigates the risk of overfitting to the limited number of training views. In addition, we have observed that overfitting primarily manifests during the later stage of training rather than the initial phase under sparse-view conditions. Based on this observation, we further proposed to progressively apply our DropGaussian scheme to the training process of 3DGS. The main contribution of the proposed method can be summarized as follows:

- We propose a simple yet powerful regularization technique, so-called DropGaussian, for rendering with sparse input views. By randomly eliminating Gaussians during the training process, DropGaussian gives the opportunity for the remaining Gaussians to be more visible with larger gradients, which make them to meaningfully contribute to the optimization process of 3DGS. This is fairly desirable to alleviate the overfitting problem occurring in sparse-view conditions.
- Through various experiments on benchmark datasets, we identified that overfitting predominantly occurs during the later stage of training rather than the initial phase with sparse input views. Based on this observation, we propose to progressively increase the ratio of dropping Gaussians during the training process. This adaptive strategy effectively mitigates the overfitting problem without unexpected impact on the rendering performance at the initial phase.

2. Related Works

In this Section, we provide a brief review of recent methods in novel view synthesis and explore various approaches specifically designed to address challenges related to sparse input views.

2.1. Novel View Synthesis

Recently, the neural radiance field (NeRF) [14] has achieved the significant progress in encoding a given 3D scene into implicit radiance fields by mapping 3D coordinates and the view direction to color and density values through a simple neural network. Numerous variants have been introduced particularly focusing on improving the rendering quality [1, 2, 25], accelerating the training speed [6, 15], and extending the model to handle more complex scenarios such as dynamic scenes [19] or unconstrained scenes [12]. Despite such extensive efforts on enhancing the capability of NeRF, most previous methods require time-consuming processes for both training and rendering, which limits their practical applicability. To address this limitation, a new approach, which is known as 3D Gaussian splatting (3DGS) [8], has been emerged. 3DGS represents a given scene by utilizing a set of 3D Gaussians and renders images via a differentiable rasterization scheme. By replacing the neural network with point-based 3D Gaussians, 3DGS remarkably enhances the efficiency of training and rendering, enabling real-time applications of 3DGS. Inspired by plentiful possibilities of 3DGS, various follow-up studies have been introduced in most recent days, e.g., improving the rendering quality [33], addressing the memory inefficiency [10, 16], and considering the temporal relationship for dynamic scenes [29].

2.2. Novel View Synthesis with Sparse-Views

Even though NeRF and 3DGS have shown surprising rendering performance, a large number of input images are required to guarantee the high-quality result. However, in real-world scenarios, only a small number of images are available (i.e., sparse input views), which leads to the performance degradation by the overfitting problem. Before the emergence of 3DGS, NeRF-based methods have attempted to resolve this problem in various ways. In the early stages, Yu *et al.* [32] employed a pre-trained CNN encoder to incorporate the contextual information into NeRF

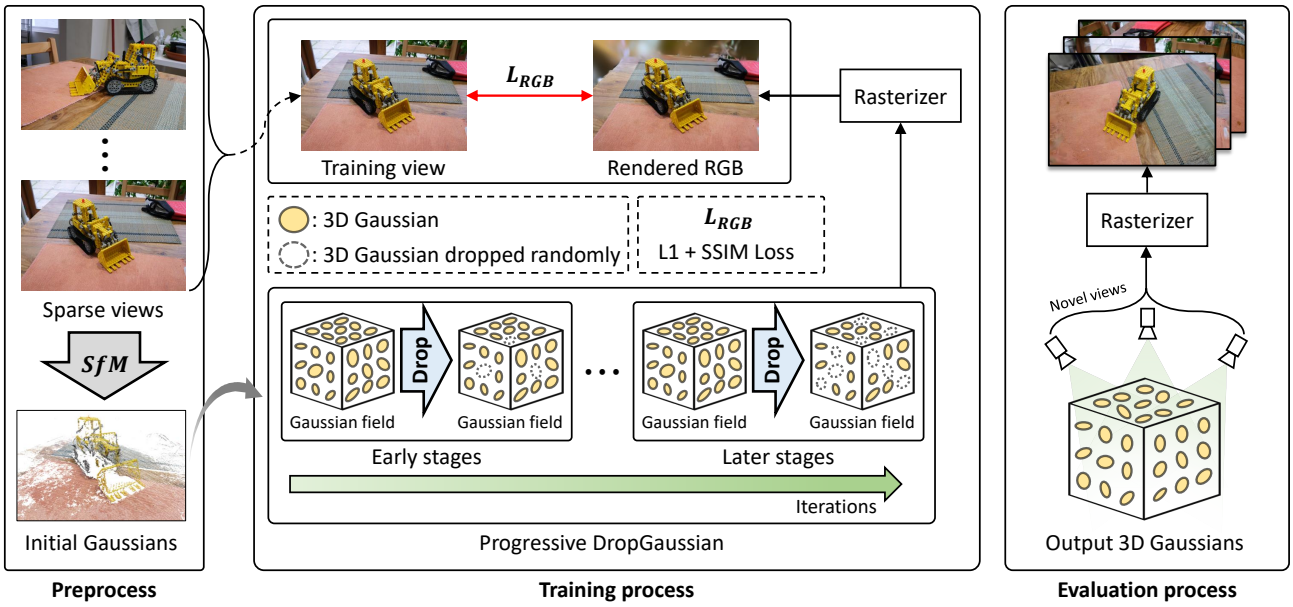


Figure 2. The overall framework of the proposed method for 3DGS in sparse-view settings. Our DropGaussian scheme improves the visibility of Gaussians even far from the camera by randomly dropping Gaussians during the training process, thereby mitigating overfitting to the limited number of training views. In contrast, during the test phase, all the Gaussians are rendered to generate high-quality RGB images, ensuring that the complete scene representation is utilized for novel view synthesis.

through transfer learning. Jain *et al.* [7] designed a semantic consistency loss by utilizing CLIP embeddings [22] to effectively handle unseen views. Furthermore, Niemeyer *et al.* [17] introduced a patch-based color and depth regularization method, making geometry and appearance across neighboring regions be consistent. Most recently, Yang *et al.* [31] suggested a simple yet powerful scheme, i.e., frequency regularization, which improves the generalization ability by adjusting the input frequency range of NeRF. Wang *et al.* [26] introduced a local depth ranking constraint for sparse input views, ensuring that the estimated depth order within local regions remains consistent with coarse observations, thereby mitigating inaccuracies in sparse scenarios.

On the other hand, several methods in 3DGS have begun to be actively studied for sparse input views. Specifically, Chung *et al.* [5] attempted to clarify the depth ambiguity by injecting the learnable parameter during training, which often occurs between different views. In a similar way, Li *et al.* [11] proposed a global-local depth normalization technique, which normalizes depth values on patch-based local scales, thereby accurately refocusing on small changes of the local depth. Instead of conducting depth-based regularization, Zhu *et al.* [36] introduced a Gaussian unpooling scheme, which is designed to generate new Gaussians by leveraging graph-based proximity scores from the nearest K neighbors. Xiong *et al.* [30] proposed to apply score distillation sampling loss [21] to the training process for refining plausible details in regions with limited coverage in training views (i.e., sparse input views) and generating more complete 3D representations. In particular, Paliwal *et al.* [18] utilized a pre-trained optical flow model [23] to regularize the pixel-wise correspondence between 3D Gaussians, which efficiently alleviates the overfitting problem by sparse input views.

3. Proposed Method

The proposed method aims to improve the updating process of Gaussian parameters in sparse-view settings. In this Section, we introduce DropGaussian, which randomly removes Gaussians during training, thereby increasing the updating opportunities for the remaining ones under sparse-view conditions thereby improving the visibility for the remaining Gaussians under sparse-view conditions. It is noteworthy that the ratio of dropping Gaussians progressively increases, which better mitigates the overfitting problem in the later stage of training. The overall framework of the proposed method is shown in Fig. 2.

3.1. Preliminaries

3D Gaussian splatting (3DGS) [8] explicitly represents a given scene by using a set of point-based 3D Gaussians. Each Gaussian is defined by a center $\mu \in \mathbb{R}^3$, a scaling factor $s \in \mathbb{R}^3$, and a rotation quaternion $q \in \mathbb{R}^4$. The basis function for the i-th Gaussian, i.e., G_i, is given by:

$$G_i(\boldsymbol{x}) = \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i)\right), \quad (1)$$

where the covariance matrix Σ_i is approximated by the combination of the scaling factor s_i and the rotation quaternion q_i, which determines size and orientation of the Gaussian in the 3D space. In addition to these geometric attributes, each Gaussian also possesses an opacity value $o_i \in \mathbb{R}$ and a K-dimensional color feature $f_i \in \mathbb{R}^K$. The color feature is typically represented by using spherical harmonic (SH) coefficients, which efficiently encode the lighting effect according to different directions. For rendering, the color of the i-th Gaussian, i.e., RGB value, is computed from these SH coefficients. The final color $C(p)$ at a given pixel p is computed by accumulating the weighted color contributions of all Gaussians, which can be formulated as follows:

$$C(p) = \sum_{i=1}^{N} c_i \alpha_i T_i, \tag{2}$$

where c_i is the RGB value derived from the color feature f_i of the i-th Gaussian. α_i denotes the product of the projected 2D Gaussian and the learned opacity value o_i of the i-th Gaussian. T_i represents the accumulated transparency along the view direction of the i-th Gaussian. N denotes the total number of Gaussians contributing to the final color of the pixel p. By following [8], T_i and α_i can be computed as follows:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j), \quad \alpha_i = o_i \cdot g_i^{2D}, \tag{3}$$

where g_i^{2D} is the 2D Gaussian, which is obtained by projecting the i-th 3D Gaussian onto the image plane.

3.2. DropGaussian

In sparse-view conditions, the transmittance T_i of 3D Gaussians, which are far from the camera and thus have a high probability of being occluded by other Gaussians, becomes relatively low as illustrated in Fig. 1(b). Since the visible range of such Gaussians is strictly limited due to a small number of input views, Gaussian attributes, e.g., scale, color, opacity, etc., are not able to be actively updated, leading to reduction of their contributions to the overall rendering process. Consequently, the gradient feedback of the optimization process in 3DGS is not sufficiently provided to the corresponding Gaussians, and it eventually results in overfitting to a small number of training views (i.e., sparse input views).

To address this problem, we propose DropGaussian, a structural regularization technique which randomly removes a set of Gaussians during the training process. Specifically, the dropping rate r is firstly defined, for example, r is set to 0.1 for 10% removal of total Gaussians. Since the cumulative opacity contributing to the color value of each pixel is probably decreased by this dropping process, we propose to apply the compensation factor to the opacity value of the remaining Gaussians as follows:

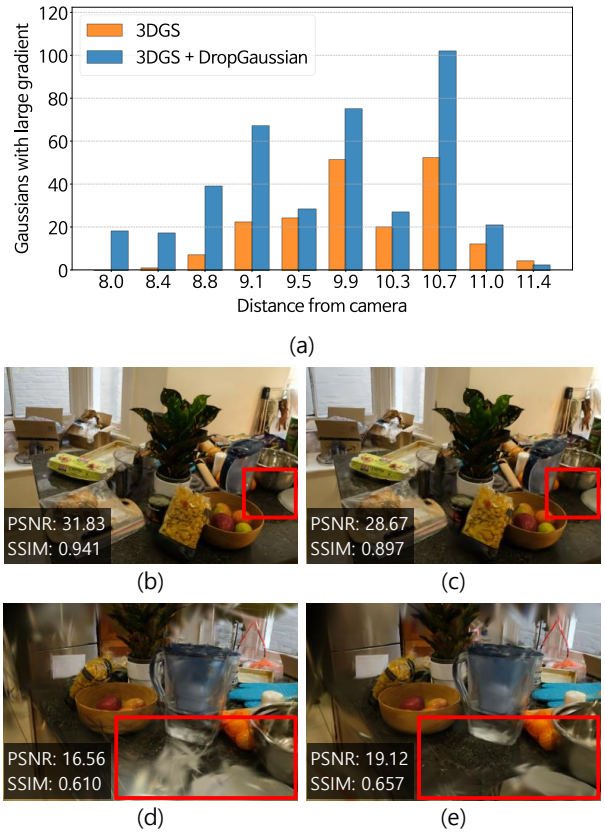


Figure 3. (a) Distribution of Gaussians having larger gradients according to the distance from camera. Note that the y-axis denotes the number of Gaussians with the gradient value, which is larger than the threshold value of densification in 3DGS. (b) Rendering results for the training view by the 3DGS. (c) Rendering results for the training view by the 3DGS with our DropGaussian scheme. (d) Rendering result for the novel view by 3DGS. (e) Rendering result for the novel view by 3DGS with our DropGaussian scheme.

$$\tilde{o}_i = M(i) \cdot o_i, \tag{4}$$

where $M(i)$ indicates the compensation factor for the i-th Gaussian, which assigns $\frac{1}{(1-r)}$ to the remaining Gaussians and 0 otherwise. It is noteworthy that the total contribution of Gaussians to the color value of each pixel is successfully maintained by the scaling effect of the compensation factor even with the dropping process. Moreover, Gaussians that are far from the camera can have large gradients by DropGaussian because their visibility is efficiently improved as shown in Fig. 1 (c).

To show the effect of our DropGaussian scheme, we further analyze the distribution of Gaussians with their gradient values according to the distance from the camera. As shown in Fig. 3 (a), Gaussians can have larger gradients by DropGaussian even though they are somewhat far from the camera. Note that we only count Gaussians with gradients

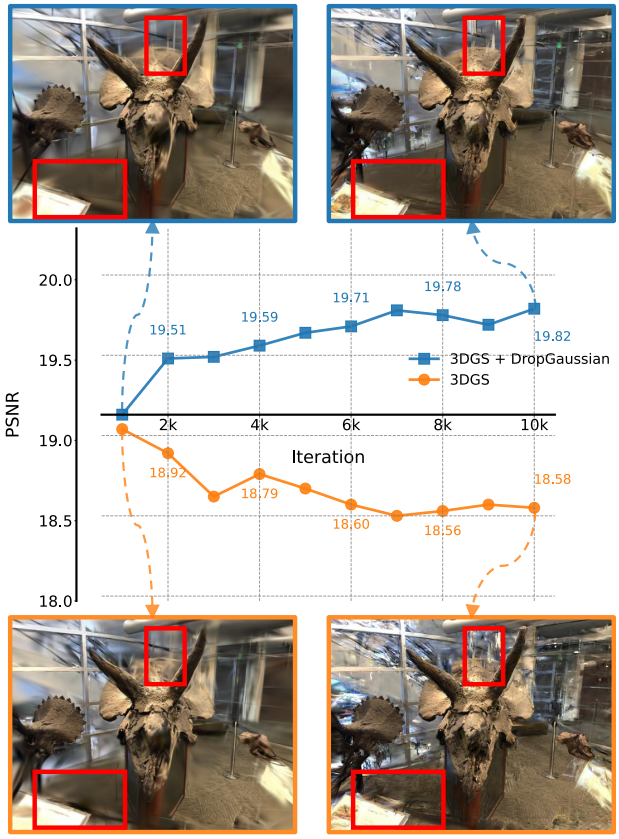


Figure 4. Changes of PSNR values during the training process on the LLFF dataset for 3DGS and 3DGS with DropGaussian. Note that rendering results for novel views at 1,000 and 10,000 iterations are also given, highlighting the intensification of overfitting in later training stages and the effectiveness of DropGaussian in mitigating this issue. The red boxes indicate regions where overfitting occurs.

whose value is larger than the threshold value for densification in 3DGS (i.e., 0.0005 in this example). This makes 3DGS to be robust to the overfitting problem when rendering novel view images under sparse-view conditions (see Fig. 3 (e)).

Furthermore, we have observed that the tendency for overfitting becomes stronger under sparse-view conditions as the training process progresses as shown in Fig. 4. To resolve this problem, we propose to adjust the dropping rate according to the current iteration index t as follows:

$$r_t = \gamma \cdot \frac{t}{t_{\text{total}}}, \tag{5}$$

where $\gamma \in \{0, 1\}$ denotes the scaling factor for the dropping rate. t_{total} is the total iterations of the training process. This progressive adjustment strategy strengthens the regularization effect as the training process progresses.

3.3. Loss Function

The proposed method is trained based on the traditional color reconstruction loss. Following 3D Gaussian splatting, the color reconstruction loss is composed of L1 loss and D-SSIM loss, which measures the structural similarity between the rendered image $\hat{I}$ and the ground-truth image I, given as [8]:

$$\mathcal{L}_{\text{color}} = \mathcal{L}_1(\hat{I}, I) + \lambda \mathcal{L}_{\text{D-SSIM}}(\hat{I}, I), \tag{6}$$

where λ denotes the weighting factor that makes a balance between the contribution of $\mathcal{L}_1$ and $\mathcal{L}_{\text{D-SSIM}}$, which is set to 0.2.

4. Experimental Results

4.1. Training

All experiments were conducted by using the PyTorch framework [20], running on an Intel E5-1650 v4@3.60GHz CPU and an NVIDIA RTX 3090Ti GPU. We employed the Adam optimizer [9] to train all model parameters, with momentum factors set to 0.9 and 0.999, respectively. The proposed method is trained for 10,000 iterations with densification performed every 100 iterations. The gradient threshold of densification is set to 5×10^{-4} as used in [36].

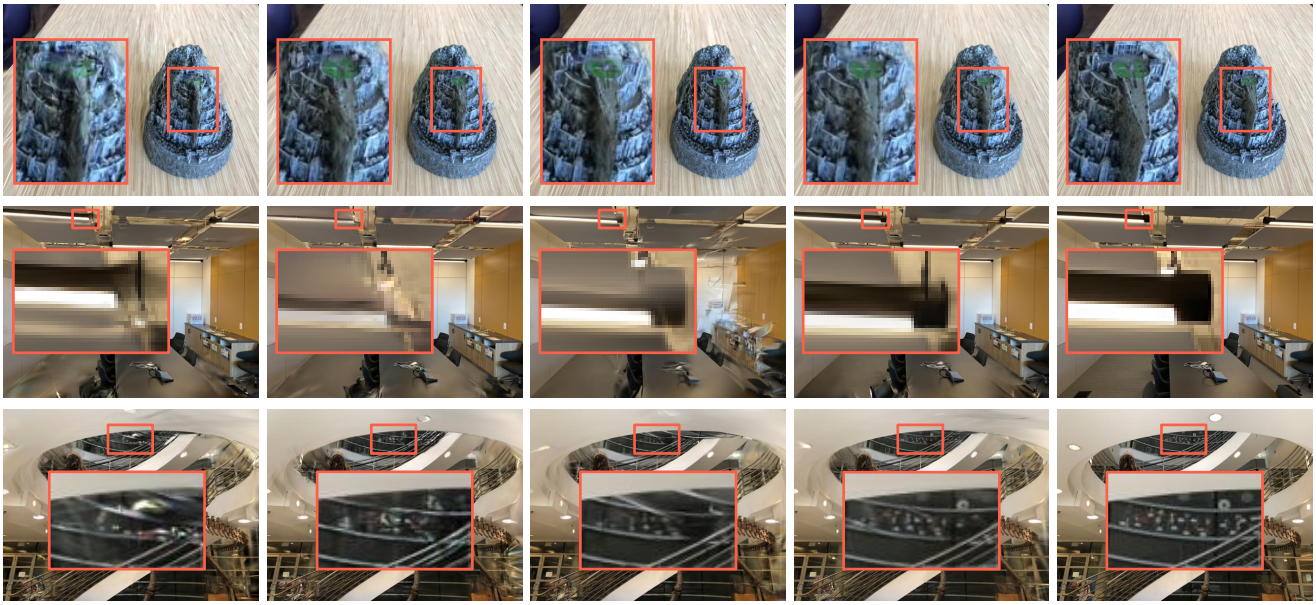
4.2. Datasets and Evaluation Metrics

Dataset. For performance evaluation of the proposed method, three representative benchmarks, i.e., LLFF [13], Mip-NeRF360 [2], and Blender [14], are employed. We follow the settings used in previous works with the same split of LLFF, Mip-NeRF360, and blender datasets, which consist of 3, 12, and 8 training views, respectively. The downsampling rates are set to 8 for both LLFF and Mip-NeRF360 while 2 is used for Blender.

Evaluation metrics. For the quantitative evaluation, we use three metrics, i.e., peak signal-to-noise ratio (PSNR), structural similarity index (SSIM) [28], and learned perceptual image patch similarity (LPIPS) [35], which have been generally adopted in this field. Specifically, PSNR measures the average peak error between the rendered image and the ground truth. SSIM computes the structural similarity based on luminance, contrast, and texture information. On the other hand, LPIPS computes the perceptual distance by utilizing learned features, which is useful to figure out underlying differences that are not reflected by traditional metrics.

4.3. Performance Evaluation

Quantitative evaluation. To demonstrate the effectiveness of the proposed method in addressing overfitting, we compare it with previous methods in rendering with sparse input views, i.e., MipNeRF [1], DietNeRF [7], RegNeRF [17], FreeNeRF [31], SparseNeRF [26], DNGaus-



(a) (b) (c) (d) (e)

Figure 5. Results of novel view rendering on the LLFF [2] dataset. (a) Results by 3DGS [8]. (b) Results by FSGS [36]. (c) Results by CoR-GS [34]. (d) Results by the proposed method. (e) GT image.

	Methods	3-view			6-view			9-view		
		PSNR(↑)	SSIM(↑)	LPIPS (↓)	PSNR(↑)	SSIM(↑)	LPIPS (↓)	PSNR(↑)	SSIM(↑)	LPIPS (↓)
NeRF-based	Mip-NeRF [1]	16.11	0.401	0.460	22.91	0.756	0.213	24.88	0.826	0.170
	DietNeRF [7]	14.94	0.370	0.496	21.75	0.717	0.248	24.28	0.801	0.183
	RegNeRF [17]	19.08	0.587	0.336	23.10	0.760	0.206	24.86	0.820	0.161
	FreeNeRF [31]	19.63	0.612	0.308	23.73	0.779	0.195	25.13	0.827	0.160
	SparseNeRF [26]	19.86	0.624	0.328	-	-	-	-	-	-
3DGS-based	3DGS [8]	19.22	0.649	0.229	23.80	0.814	0.125	25.44	0.860	0.096
	DNGaussian [11]	19.12	0.591	0.294	22.18	0.755	0.198	23.17	0.788	0.180
	FSGS [36]	20.43	0.682	0.248	24.09	0.823	0.145	25.31	0.860	0.122
	CoR-GS [34]	20.45	0.712	0.196	24.49	0.837	0.115	26.06	0.874	0.089
	Ours	20.76	0.713	0.200	24.74	0.837	0.117	26.21	0.874	0.088

Table 1. Performance comparisons of sparse-view synthesis on LLFF dataset [13]. The best, second-best, and third-best entries are marked in red, orange, and yellow, respectively.

sian [11], FSGS [36], and CoR-GS [34]. First of all, the performance comparison on the LLFF dataset is shown in Table 1. As can be seen, the proposed method shows the meaningful improvement for the rendering performance with a very simple operation. Specifically, the proposed method achieves the highest PSNR of 20.76 in the 3-view setting, while surpassing all NeRF-based and 3DGS-based methods. For both 6-view and 9-view settings, our approach still achieves the competitive performance compared to the state-of-the-art methods (e.g., CoR-GS [34]) without any increase in the computational complexity.

The performance comparison on the Mip-NeRF360 dataset is also shown in Table 2. It is noteworthy that our approach attains 23.92 (PSNR), 0.755 (SSIM), and 0.242 (LPIPS), which significantly outperforms the state-of-the-art methods by a meaningful margin. Finally, we also evaluate the rendering performance on the Blender dataset, and the corresponding result is shown in Table 3. The proposed method achieves the highest PSNR of 25.42 while the performance for other two metrics are slightly dropped compared to the best scores. Even though the proposed method always shows the best performance for all the metrics, it is still effective and competitive for rendering with sparse view inputs without requiring any additional module or algorithm.

Besides, we compare the proposed method with feed-forward 3DGS methods, such as pixelSplat [3], MVSplat [4], and FreeSplat [27], on the Replica dataset (see Table 4). While these pre-trained feed-forward models can provide faster inference, our method still demonstrates high visual quality under sparse-view inputs.

Qualitative evaluation. Furthermore, the qualitative comparison of the proposed method with FSGS [36], CoR-GS [34], and 3DGS on the LLFF dataset is presented in Fig. 5. The results demonstrate the effectiveness of our approach in forward-facing scenes, particularly in achiev-

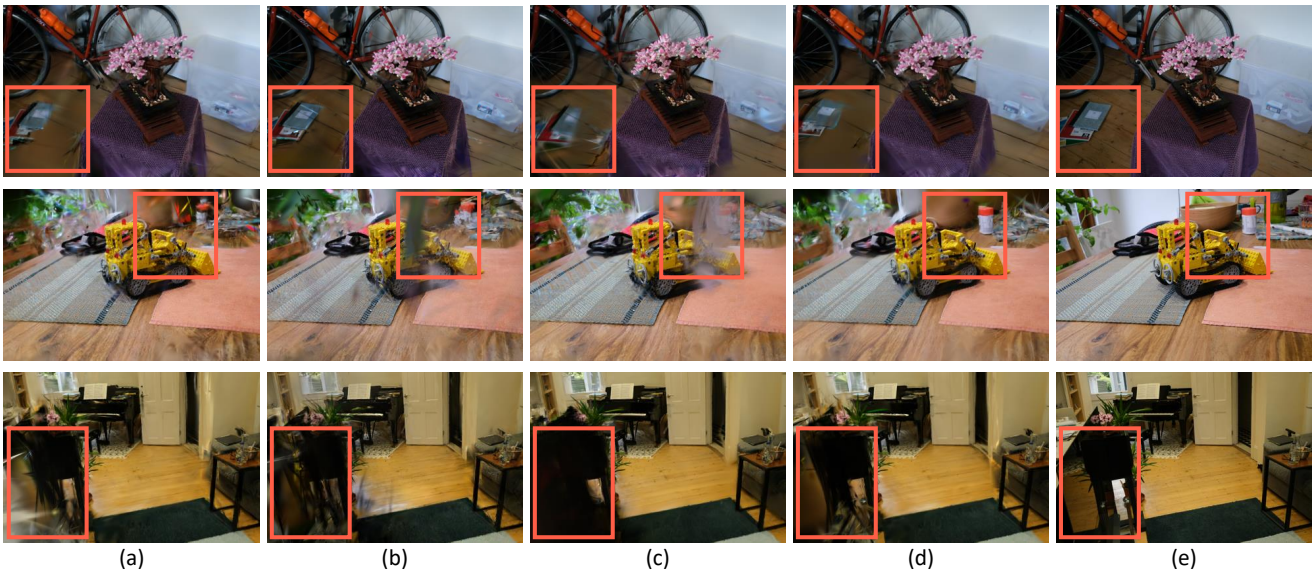


Figure 6. Results of novel view rendering on the Mip-NeRF360 [2] dataset. (a) Results by 3DGS [8]. (b) Results by FSGS [36]. (c) Results by CoR-GS [34]. (d) Results by the proposed method. (e) GT image.

Methods	12-view			24-view		
	PSNR(↑)	SSIM(↑)	LPIPS (↓)	PSNR(↑)	SSIM(↑)	LPIPS (↓)
3DGS [8]	18.52	0.523	0.415	22.80	0.708	0.276
FSGS [36]	18.80	0.531	0.418	23.70	0.745	0.230
CoR-GS [34]	19.52	0.558	0.418	23.39	0.727	0.271
Ours	19.74	0.577	0.364	24.13	0.762	0.225

Table 2. Performance comparisons of sparse-view synthesis on Mip-NeRF360 dataset [2]. The best, second-best, and third-best entries are marked in red, orange, and yellow, respectively.

ing high precision and artifact-free renderings compared to baseline methods. In the first row of Fig. 5, our method consistently outperforms other approaches in rendering forward-facing scenes with greater precision. While other methods exhibit visible inaccuracies in capturing fine details and maintaining structural coherence, our approach achieves superior fidelity, preserving intricate scene features and producing more realistic outputs. In the second row, the robustness of the proposed method in mitigating overfitting is further demonstrated. FSGS and CoR-GS exhibit prominent artifacts that compromise the quality and realism of the renderings. In contrast, our method effectively avoids these artifacts, maintaining clean and accurate reconstructions even in challenging regions. Rendering results from novel views by the proposed method are also shown in Fig. 7. These qualitative results reinforce the effectiveness of our method in addressing overfitting and ensuring robust performance across different datasets and scene types, showcasing its versatility and reliability in a wide range of rendering tasks.

Figure 7. Results of novel view rendering on the Blender dataset [14] by the proposed method.

4.4. Limitations and Future Work

While the proposed method demonstrates significant improvements in mitigating overfitting and enhancing rendering quality in sparse-view 3D Gaussian splatting (3DGS), there are certain limitations that require further investigation. These limitations highlight opportunities for future research and practical enhancements. The reliance on hyperparameters, such as the scaling factor γ for the dropping rate r, introduces sensitivity to dataset-specific tuning. While γ plays a crucial role in progressively adjusting the dropping rate to mitigate overfitting, its optimal value can vary depending on the dataset and task. Future work could explore adaptive mechanisms to dynamically adjust γ during training, reducing the need for manual fine-tuning and improving generalization across diverse datasets.

	Methods	PSNR(↑)	SSIM(↑)	LPIPS (↓)
NeRF-based	Mip-NeRF [1]	20.89	0.830	0.168
	DietNeRF [7]	22.50	0.823	0.124
	RegNeRF [17]	23.86	0.852	0.105
	FreeNeRF [31]	24.26	0.883	0.098
	SparseNeRF [26]	24.04	0.876	0.113
3DGS-based	3DGS [8]	21.56	0.847	0.130
	DNGaussian [11]	24.31	0.886	0.088
	FSGS [36]	24.64	0.895	0.095
	CoR-GS [34]	24.43	0.896	0.084
	Ours	25.42	0.888	0.089

Table 3. Performance comparisons of sparse-view synthesis on Blender dataset [14]. The best, second-best, and third-best entries are marked in red, orange, and yellow, respectively.

Methods	Type	PSNR (↑)	SSIM (↑)	LPIPS (↓)
pixelSplat [3]	Feed-forward	26.24	0.829	0.229
MVSplat [4]	Feed-forward	26.16	0.840	0.173
FreeSplat [27]	Feed-forward	26.98	0.848	0.171
Ours	Optimization	27.76	0.866	0.179

Table 4. Performance comparison with feed-forward methods on the Replica dataset (2-view) [24]. The best, second-best, and third-best entries are marked in red, orange, and yellow, respectively.

4.5. Ablation Study

In this subsection, we check the effect of the change in dropping rates and the progressive adjustment strategy. The performance for all the experiments in this subsection is evaluated on the LLFF [13] dataset with the 3-view setting. Note that the scale factor of the dropping rate becomes the dropping rate when the progressive adjustment strategy is not used. Specifically, compared to the use of the fixed dropping rate, our progressive adjustment strategy efficiently improves the rendering performance regardless of the value of the dropping rate as shown in Table 5. For the fixed dropping rates, the performance degradation was observed as the ratio of dropping Gaussians increased from 0.1 to 0.3, which indicates the adverse effect of overly aggressive Gaussian removal. One interesting point is that this reversal highlights the effectiveness of progressively increasing the dropping rate in alleviating the overfitting problem, particularly during the later stage of training. The best performance is achieved when using 0.2 for the scale factor of the dropping rate, thus $\gamma = 0.2$ is our default setting. These findings demonstrate the utility of our progressive adjustment of the dropping rate for reducing overfitting while improving the rendering quality. Furthermore, we observe that randomly dropping Gaussian primitives is more effective than selectively dropping them. Since selective approaches rely on metrics like gradient magnitude and distance, they have a potential risk repeatedly discarding Gaussians that are critical for reconstructing scenes under sparse-view conditions. The corresponding result is shown in Table 6. In the domain of optimization, L1 regularization is commonly used to prune elements that exhibit a lower correlation with the objective function. However, this approach may permanently remove Gaussians, whereas our method only temporarily deactivates Gaussians. That is, strongly correlated Gaussians can re-engage during later iterations. This property efficiently improves the performance compared to L1 regularization as shown in Table 7.

Scaling factor (γ)	Progressive	PSNR (↑)	SSIM (↑)	LPIPS (↓)
-	-	19.22	0.649	0.229
0.1	✗	20.28	0.701	0.209
0.2	✗	20.16	0.700	0.216
0.3	✗	20.15	0.691	0.223
0.1	✓	20.29	0.702	0.206
0.2	✓	20.56	0.708	0.207
0.3	✓	20.48	0.707	0.212

Table 5. Ablation study on the LLFF dataset (3-view). The best, second-best, and third-best entries are marked in red, orange, and yellow, respectively.

Methods	Metrics	PSNR (↑)	SSIM (↑)	LPIPS (↓)
Random	-	20.76	0.713	0.200
Selective	Gradient	19.62	0.682	0.214
Selective	Distance	17.49	0.582	0.273

Table 6. Ablation study according to dropping schemes. The best, second-best, and third-best entries are marked in red, orange, and yellow, respectively.

Methods	Metrics	PSNR (↑)	SSIM (↑)	LPIPS (↓)
Dropping	-	20.76	0.713	0.200
L1 reg.	Gradient	19.97	0.690	0.211
L1 reg.	Distance	19.73	0.681	0.213

Table 7. Ablation study according to regularization schemes. The best, second-best, and third-best entries are marked in red, orange, and yellow, respectively.

5. Conclusion

In this paper, we introduce a simple yet powerful method for sparse-view 3DGS. The key idea of the proposed method, so-called DropGaussian, is to mitigate the overfitting problem by randomly removing 3D Gaussians during the training process. By simple dropping operations, Gaussians, which are far from the camera in sparse-view conditions, can have larger gradients while improving their visibility. This is fairly desirable to mitigate the overfitting problem in rendering with sparse input views. Moreover, we propose to progressively apply our DropGaussian to the training process for further enhancing the visual quality of the rendering result.

Acknowledgments. This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2023-NR076462) and Institute of Information Communications Technology Planning Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2018-0-00207, RS-2018-II180207, Immersive Media Research Laboratory).

References

[1] Jonathan T Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 5855–5864, 2021.

[2] Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 5470–5479, 2022.

[3] David Charatan, Sizhe Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 19457–19467, 2024.

[4] Yuedong Chen, Haofei Xu, Chuanxia Zheng, Bohan Zhuang, Marc Pollefeys, Andreas Geiger, Tat-Jen Cham, and Jianfei Cai. Mvsplat: Efficient 3d gaussian splatting from sparse multi-view images. In *Proc. Eur. Conf. Comput. Vis.*, pages 370–386, 2024.

[5] Jaeyoung Chung, Jeongtaek Oh, and Kyoung Mu Lee. Depth-regularized optimization for 3d gaussian splatting in few-shot images. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 811–820, 2024.

[6] Sara Fridovich-Keil, Alex Yu, Matthew Tancik, Qinhong Chen, Benjamin Recht, and Angjoo Kanazawa. Plenoxels: Radiance fields without neural networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 5501–5510, 2022.

[7] Ajay Jain, Matthew Tancik, and Pieter Abbeel. Putting nerf on a diet: Semantically consistent few-shot view synthesis. In *Proc. Int. Conf. Comput. Vis.*, pages 5885–5894, 2021.

[8] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023.

[9] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proc. Int. Conf. Learn. Represent.*, pages 1–13, 2015.

[10] Joo Chan Lee, Daniel Rho, Xiangyu Sun, Jong Hwan Ko, and Eunbyung Park. Compact 3d gaussian representation for radiance field. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 21719–21728, 2024.

[11] Jiahe Li, Jiawei Zhang, Xiao Bai, Jin Zheng, Xin Ning, Jun Zhou, and Lin Gu. Dngaussian: Optimizing sparse-view 3d gaussian radiance fields with global-local depth normalization. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 20775–20785, 2024.

[12] Ricardo Martin-Brualla, Noha Radwan, Mehdi SM Sajjadi, Jonathan T Barron, Alexey Dosovitskiy, and Daniel Duckworth. Nerf in the wild: Neural radiance fields for unconstrained photo collections. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 7210–7219, 2021.

[13] Ben Mildenhall, Pratul P Srinivasan, Rodrigo Ortiz-Cayon, Nima Khademi Kalantari, Ravi Ramamoorthi, Ren Ng, and Abhishek Kar. Local light field fusion: Practical view synthesis with prescriptive sampling guidelines. *ACM Trans. Graph.*, 38(4):1–14, 2019.

[14] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *Proc. Eur. Conf. Comput. Vis.*, pages 405–421, 2020.

[15] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. Graph.*, 41(4):1–15, 2022.

[16] Simon Niedermayr, Josef Stumpfegger, and Rüdiger Westermann. Compressed 3d gaussian splatting for accelerated novel view synthesis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 10349–10358, 2024.

[17] Michael Niemeyer, Jonathan T Barron, Ben Mildenhall, Mehdi SM Sajjadi, Andreas Geiger, and Noha Radwan. Regnerf: Regularizing neural radiance fields for view synthesis from sparse inputs. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 5480–5490, 2022.

[18] Avinash Paliwal, Wei Ye, Jinhui Xiong, Dmytro Kotovenko, Rakesh Ranjan, Vikas Chandra, and Nima Khademi Kalantari. Coherentgs: Sparse novel view synthesis with coherent 3d gaussians. In *Proc. Eur. Conf. Comput. Vis.*, pages 19–37, 2024.

[19] Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T Barron, Sofien Bouaziz, Dan B Goldman, Ricardo Martin-Brualla, and Steven M Seitz. Hypernerf: a higher-dimensional representation for topologically varying neural radiance fields. *ACM Trans. Graph.*, 40(6):1–12, 2021.

[20] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. In *Adv. Neural Inform. Process. Syst.*, pages 1–4, 2017.

[21] Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. In *Proc. Int. Conf. Learn. Represent.*, 2023.

[22] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. pages 8748–8763, 2021.

[23] Xiaoyu Shi, Zhaoyang Huang, Dasong Li, Manyuan Zhang, Ka Chun Cheung, Simon See, Hongwei Qin, Jifeng Dai, and Hongsheng Li. Flowformer++: Masked cost volume autoencoding for pretraining optical flow estimation. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 1599–1610, 2023.

[24] Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J Engel, Raul Mur-Artal, Carl Ren, Shobhit Verma, et al. The replica dataset: A digital replica of indoor spaces. *arXiv preprint arXiv:1906.05797*, 2019.

[25] Dor Verbin, Peter Hedman, Ben Mildenhall, Todd Zickler, Jonathan T Barron, and Pratul P Srinivasan. Ref-nerf: Structured view-dependent appearance for neural radiance fields. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 5481–5490, 2022.

[26] Guangcong Wang, Zhaoxi Chen, Chen Change Loy, and Ziwei Liu. Sparsenerf: Distilling depth ranking for few-shot

novel view synthesis. In *Proc. Int. Conf. Comput. Vis.*, pages 9065–9076, 2023.

[27] Yunsong Wang, Tianxin Huang, Hanlin Chen, and Gim Hee Lee. Freesplat: Generalizable 3d gaussian splatting towards free view synthesis of indoor scenes. *Adv. Neural Inform. Process. Syst.*, 37:107326–107349, 2025.

[28] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Trans. Image Process.*, 13(4): 600–612, 2004.

[29] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4d gaussian splatting for real-time dynamic scene rendering. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 20310–20320, 2024.

[30] Haolin Xiong, Sairisheek Muttukuru, Rishi Upadhyay, Pradyumna Chari, and Achuta Kadambi. Sparsegs: Real-time 360° sparse view synthesis using gaussian splatting. *arXiv preprint arXiv:2312.00206*, 2023.

[31] Jiawei Yang, Marco Pavone, and Yue Wang. Freenerf: Improving few-shot neural rendering with free frequency regularization. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 8254–8263, 2023.

[32] Alex Yu, Vickie Ye, Matthew Tancik, and Angjoo Kanazawa. pixelnerf: Neural radiance fields from one or few images. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 4578–4587, 2021.

[33] Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 19447–19456, 2024.

[34] Jiawei Zhang, Jiahe Li, Xiaohan Yu, Lei Huang, Lin Gu, Jin Zheng, and Xiao Bai. Cor-gs: sparse-view 3d gaussian splatting via co-regularization. In *Proc. Eur. Conf. Comput. Vis.*, pages 335–352, 2024.

[35] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, pages 586–595, 2018.

[36] Zehao Zhu, Zhiwen Fan, Yifan Jiang, and Zhangyang Wang. Fsgs: Real-time few-shot view synthesis using gaussian splatting. In *Proc. Eur. Conf. Comput. Vis.*, pages 145–163, 2024.